



ICS3U Resource Book

Revised August 2013

2.1 Basic Arithmetic Operations

Many arithmetic expressions in Java look much like ordinary arithmetic expressions that you would see in a mathematics text. The four basic arithmetic operations are shown in the following table.

Operation	Java Operator
Addition	+
Subtraction	-
Multiplication	*
Division	/

The normal precedence of operations applies, with multiplication and division being done (from left to right) before addition and subtraction (again, from left to right). Parentheses (but not square or brace brackets) can be used to change the order of evaluation. The multiplication symbol cannot be omitted as it sometimes is in mathematics. Operators can be written with or without extra spaces around them. Java also permits the use of *unary plus* and *unary minus* in which a + or - is placed in front of an expression. The unary plus and unary minus operators have a higher precedence than any of the other arithmetic operators.

Example 1

To evaluate the expression $+4 + 3 * (5 - 6/2)$, Java would proceed as follows:

$$\begin{aligned}
 +4 + 3 * (5 - 6/2) &\Rightarrow 4 + 3 * (5 - 6/2) \\
 &\Rightarrow 4 + 3 * (5 - 3) \\
 &\Rightarrow 4 + 3 * 2 \\
 &\Rightarrow 4 + 6 \\
 &\Rightarrow 10
 \end{aligned}$$

We use the symbol $\Rightarrow$ rather than $=$ in evaluations of expressions to avoid confusion with the use of $=$ as an assignment operator. You may find it useful to read $\Rightarrow$ as “gives” or “evaluates to”.

In Example 1, all the values were of type `int` and the result was also of type `int`. It is generally true that operations on values of the same type produce results of that type. In particular, if we divide two integers, the result is the integral quotient with any remainder being ignored. The values of the divisor or dividend may be either positive or negative. The sign of the quotient follows the usual rules of division.

Example 2

- | | |
|------------------------------|-----------------------------------|
| (a) $15/3 \Rightarrow 5$ | (b) $13/4 \Rightarrow 3$ |
| (c) $9/5 \Rightarrow 1$ | (d) $7/9 \Rightarrow 0$ |
| (e) $27L/10L \Rightarrow 2L$ | (f) $9999L/10000L \Rightarrow 0L$ |
| (g) $7/(-3) \Rightarrow -2$ | (h) $-15/4 \Rightarrow -3$ |
| (i) $-5/(-6) \Rightarrow 0$ | (j) $(-9)/(-5) \Rightarrow 1$ |

Whenever integer and floating point values are mixed in an expression, the floating point values are contagious so that the result is always a floating point value. In addition, if different integer types are mixed, the result is that of the widest type. The same is true if floating point types are mixed: the result of combining `float` and `double` values is always `double`.

Example 3

- | | |
|----------------------------------|---------------------------------------|
| (a) $5 + 2.0 \Rightarrow 7.0$ | The result is a <code>double</code> . |
| (b) $3 * 4L \Rightarrow 12L$ | The result is a <code>long</code> . |
| (c) $2.5f + 2.5 \Rightarrow 5.0$ | The result is a <code>double</code> . |

If conversions are necessary, Java does not convert all the original values in an expression prior to computing the result. Instead, it converts values or intermediate results as necessary as the computation proceeds.

Example 4

- (a) $1+1.0/2 \Rightarrow 1+0.5 \Rightarrow 1.5$
- (b) $1.0+1/2 \Rightarrow 1.0+0 \Rightarrow 1.0$
- (c) $3/4*2.0 \Rightarrow 0*2.0 \Rightarrow 0.0$

In mathematics, division by zero is undefined. If we are dividing one integer by another and the divisor has the value zero, then Java objects to this by *throwing an exception*. Many types of exceptions can be thrown by Java, depending on the error. For this error, Java throws an `ArithmeticException`. Throwing an exception has been likened to throwing up your hands and saying “I can’t cope with this”. If any exception is thrown during the execution of a program, then the programmer can write instructions to *catch* the exception. Appendix E contains notes on handling exceptions gracefully. For now, if a program contains this error, Java will throw an `ArithmeticException` and the program will terminate prematurely after printing a message stating the form of the exception.

If either operand of a division is a floating point value and an attempt is made to divide by zero, Java does *not* throw an exception. To handle results of such an operation, Java has three special floating point values: positive infinity, negative infinity, and NaN (not a number). The result positive infinity is produced by attempting to divide a positive value by zero while negative infinity is produced by attempting to divide a negative value by zero. These two values can be used in subsequent calculations. For example, adding a non-infinite value to positive infinity produces positive infinity. The value NaN is produced if we attempt to divide zero by zero. Attempts to print expressions having these values produce the strings `Infinity`, `-Infinity`, and `NaN`.

Example 5

- | | |
|----------------|----------------------------------|
| (a) $1.0/0$ | Result is <code>Infinity</code> |
| (b) $-2.5/0.0$ | Result is <code>-Infinity</code> |

(c) `0/0.0`

Result is NaN

Another useful arithmetic operator is written as `%`. The expression `m % n` gives the remainder after `m` is divided by `n`. We sometimes refer to `m % n` as `m modulo n` (or simply `m mod n`). If either operand is negative, the value of `m % n` is found by finding the value of `|m| modulo |n|` and then taking the sign of `m`. The `%` operator is usually used with integers but, in Java, it can also be used with floating point values. If the second operand is zero, then Java throws an exception for integer values but the expression evaluates to NaN for floating point values. In expressions with a number of different operators, the `%` operator has the same precedence as multiplication and division.

Example 6

(a) `7 % 3` $\Rightarrow$ 1(b) `12 % 15` $\Rightarrow$ 12(c) `-20 % 7` $\Rightarrow$ -6(d) `5.9 % 1.2` $\Rightarrow$ 1.1(e) `8 % 0`Throws an `ArithmeticException`(f) `0.8 % 0.0` $\Rightarrow$ NaN

Exercises 2.1

1. Evaluate each valid expression and state the type of the result. If the expression is invalid, give the reason. Use a decimal point in your answer if the result is a floating point value (either `float` or `double`). For example, use 3.0 rather than 3 to write a floating point result whose value is three.

- | | |
|-------------------------|-------------------------|
| (a) $2 + 7 / 2$ | (b) $5(6 + 3.0)$ |
| (c) $2e02 - 02$ | (d) $3 * 5 / 2$ |
| (e) $3.0 * 5 / 2$ | (f) $2 * (3 / 4L)$ |
| (g) $2.0 + 7 / (3 / 4)$ | (h) $3f * 4 / 5$ |
| (i) $35 / (5 / 4)$ | (j) $4 - [6 - (2 - 7)]$ |
| (k) $3 / 4 * 2.0$ | (l) $3 / (4 * 2.0)$ |

2. Evaluate.

- | | |
|------------------------|-------------------------|
| (a) $17 \% 5$ | (b) $23 \% 10$ |
| (c) $20 / 3 + 20 \% 3$ | (d) $-10 \% 2 + 1 / 2$ |
| (e) $-7\%(-2)/5$ | (f) $2.7\%4$ |
| (g) $7 \% 1.5$ | (h) $2L + 5.7f \% 1.2f$ |
| (i) $1.5 \% (3/4)$ | (j) $2 - 5/0.0$ |
| (k) $(-7)\%(-3)$ | (l) $(-5)\%(-1.5)$ |

3. Delete any unnecessary parentheses.

- | |
|---|
| (a) $((a * b) / (c + d))$ |
| (b) $((a * b) - (c \% d))$ |
| (c) $((a - b) \% c) / (d * (e + f))$ |
| (d) $((a * b) - (c / d) - (e / (f / g)))$ |

4. Write as Java expressions.

- | | |
|------------------------------|---|
| (a) $\frac{x}{2} + 4(y - 3)$ | (b) $x^3 + y^3$ |
| (c) $2\frac{x+1}{y-2}$ | (d) $\frac{\frac{x}{y} + z}{x + \frac{y}{z}}$ |

5. Use the symbols $+$, $-$, $*$, $/$, $\%$, $($, and $)$ as often as you wish, together with the numerals 1, 8, 6, and 7, each used once, in the given order, to create Java expressions whose values are 0, 1, ..., 10.

As an example, to obtain the value zero, we could write

$$1 / 8 + 6 / 7 \Rightarrow 0 + 0 \Rightarrow 0$$

2.2 Assigning and Printing Expressions

In the previous chapter, we saw how we could give values to variables using assignment statements of the form

`<identifier> = <value>;`

As you might expect, assignment statements can be extended to give values of expressions to variables by using statements of the form

`<identifier> = <expression>;`

In carrying out such assignments, the expression on the right is evaluated and then the resulting value is stored in the memory location of the variable on the left. Of course, this will destroy any previous value that may have been stored in that location.

The type of the expression on the right and that of the variable on the left are usually identical but, if they are not, then the rules that we saw in the last chapter for making conversions apply here also.

If we are working with values of various types, we may want to use the operation of performing a *cast* that we studied in the last chapter. A cast operator is applied to the value to the immediate right of the cast. A cast operator has a higher precedence than either `*`, `/`, or `%`. Thus, if it is combined with other operations and we want it to apply to more than one value in an expression, we should use parentheses.

Example 1

- (a) `(int) 5.7 + 8.9` $\Rightarrow$ `5 + 8.9` $\Rightarrow$ `13.9` (a double value)
- (b) `(int) 5.7 + (int) 8.9` $\Rightarrow$ `5 + 8` $\Rightarrow$ `13` (an int value)
- (c) `(int) (5.7 + 8.9)` $\Rightarrow$ `(int) (14.6)` $\Rightarrow$ `14` (an int value)
- (d) `(double) (1/2)` $\Rightarrow$ `(double) 0` $\Rightarrow$ `0.0` (a double value)
- (e) `(double) 5/0` $\Rightarrow$ `5.0/0` (produces positive infinity)
- (f) `(double) (5/0)` (throws an `ArithmeticException`)

We can use the `print` or `println` methods to print values of expressions, with or without accompanying strings. In using arithmetic expressions as parts of the argument to `print` or `println`, we must be careful of the fact that the operator `+` has different meanings for string expressions (where it means concatenation) and numerical expressions (where it means addition).¹ Whichever way we use `+`, it has the same precedence as the arithmetic operator and if there is more than one occurrence of `+` in an expression, evaluation takes place from left to right unless parentheses are used to alter the order of evaluation. If `+` is used between a string and a primitive value, the primitive value is automatically converted to a string and then the two strings are concatenated. Using these rules, Java forms a single string argument for `print` or `println` and then prints this string. The next example illustrates the use of `+` in a variety of contexts.

Example 2

Suppose that the following declarations have been made:

```
int m = 3, n = 4;
```

The following table shows the output of a variety of statements involving these variables. Can you explain each line of output?

Statement	Output
<code>System.out.println("m: " + m);</code>	<code>m: 3</code>
<code>System.out.println(m + n);</code>	<code>7</code>
<code>System.out.println("m" + "n");</code>	<code>mn</code>
<code>System.out.println("m + n: " + m + n);</code>	<code>m + n: 34</code>
<code>System.out.println("m + n: " + (m + n));</code>	<code>m + n: 7</code>
<code>System.out.println("mn: " + m * n);</code>	<code>mn: 12</code>
<code>System.out.println(m + n + "m + n");</code>	<code>7m + n</code>
<code>System.out.println(m + n + (m + n));</code>	<code>14</code>

¹Since the `+` operator has different meanings in different contexts, we say that the operator is *overloaded*. Although many kinds of operator overloading are seen in some programming languages, this is the only form that it takes in Java.

Exercises 2.2

- Find the value of each expression. Use a decimal point in your answer if the result is a floating point value.

- | | |
|--------------------------------------|---------------------------------------|
| (a) <code>(int) 1.8 * 0.6</code> | (b) <code>(int)(2/0.9)</code> |
| (c) <code>6 / 5 % (double) 2</code> | (d) <code>2*(double)(7/4)</code> |
| (e) <code>(int)6.3 - (int)4.7</code> | (f) <code>(int)6.3 - 4.7</code> |
| (g) <code>(int)(6.3 - 4.7)</code> | (h) <code>(double) 3 * 5 / 0</code> |
| (i) <code>(float)2*3 + 0.1</code> | (j) <code>(double) (7 % 4 / 3)</code> |

- Assume that a program contains the following declarations.

```
int i = 5, j = 2;
```

What will be printed by each statement?

- `System.out.println(i + j + 2);`
- `System.out.println(i + " + " + j + 2);`
- `System.out.println(i * j + 2);`
- `System.out.println("i * j " + 2);`

- Assume that a program contains the following declarations.

```
int i = 4, j = 7;
```

What will be printed by each statement?

- `System.out.println("i " + i);`
- `System.out.println(i + j);`
- `System.out.println("i + j -> " + i + j);`
- `System.out.println(i + j + " <- i + j");`
- `System.out.println("i * j -> " + i * j);`

2.3 Increment and Decrement Operators

In addition to the basic arithmetic operators, Java has two useful operators that increase or decrease the value of a variable by one. The *increment operator*, `++`, adds one to a variable while the *decrement operator*, `--`, subtracts one from a variable. Both operators can be used in two forms: *prefix* form, with the operator preceding its operand and *postfix* form, with the operator following its operand.

Example 1

The statements

```
n++; and ++n;
```

both have the same effect as the statement

```
n = n + 1;
```

The statements

```
n--; and --n;
```

both have the same effect as the statement

```
n = n - 1;
```

The difference between the prefix and postfix forms only appears when these operators are combined with other operators. In this text, we will never use expressions that combine increment or decrement operators with any others so, if you do not read the rest of this section, it will not cause any difficulties. If, however, you are curious about the difference, read on.

When an expression containing an increment or decrement operator is encountered, two things occur: the value of a variable is increased or decreased and the expression is evaluated. The difference between prefix and postfix forms is the order in which these operations are performed:

1. If a variable has a *prefix* operator, the variable is incremented or decremented first and this new value is used as the value of the expression.

2. If a variable has a *postfix* operator, the variable is incremented or decremented after the old value has been used as the value of the expression.

Increment and decrement operators have a higher precedence than any of the other arithmetic operators. When combined with other arithmetic operators, increments and decrements are performed first unless parentheses change the order of operations.

Example 2

Suppose that we have the declaration:

```
int i = 2, j = 3;
```

If we then write the statement

```
i = (j++ + 4) * ++j;
```

evaluation proceeds as outlined in the following table:

Evaluation Stage	Expression Value	i	j
Start	$i = (j++ + 4) * ++j$	2	3
Evaluate $j++$, increment j	$i = (3 + 4) * ++j$	2	4
Increment j , evaluate $++j$	$i = (3 + 4) * 5$	2	5
Add 3 plus 4	$i = 7 * 5$	2	5
Multiply 7 by 5	$i = 35$	2	5
Assign 35 to i	35	35	5

The final values of i and j are 35 and 5 respectively.

Although it is perfectly correct to write expressions like those in Example 2, we do not recommend it. In general, we suggest that you only use these operators by themselves. This may involve a bit more writing but it will likely lead to greater clarity in your programs.

Exercises 2.3

1. Suppose that the following declarations have been made:

```
int i = 3, j = 2, k = 5;
```

Using these starting values in each part, find the value of each variable after the given statements have been executed.

- | | |
|--|--|
| (a) <code>i++;</code>
<code>j = --k;</code> | (b) <code>++j;</code>
<code>k = i++ + --j;</code> |
| (c) <code>j = ++i * i++;</code> | (d) <code>k = i++/j--;</code> |
| (e) <code>i = ++k * j--;</code> | (f) <code>k = (2 + i++)/(3 + ++j);</code> |

2. Suppose that the following declaration has been made:

```
int i = 0, j = 3;
```

Using these starting values in each part, find the value of each variable after the given statements have been executed.

- | | |
|-------------------------------------|-------------------------------------|
| (a) <code>i = ++j + j;</code> | (b) <code>i = j + j++;</code> |
| (c) <code>i = ++j*2 + ++j*3;</code> | (d) <code>i = ++j*3 + ++j*2;</code> |
| (e) <code>j = 3*j-- + 2*j--;</code> | (f) <code>j = 2*j-- + 3*--j;</code> |

3. Rewrite as a single statement.

- | | |
|---|---|
| (a) <code>x++;</code>
<code>y = x - z;</code>
<code>z++;</code> | (b) <code>++x;</code>
<code>z = x*y;</code>
<code>--z;</code> |
|---|---|

4. Rewrite as a sequence of three statements.

- | | |
|---------------------------------|---------------------------------|
| (a) <code>m = n++ - p--;</code> | (b) <code>p = ++n - m--;</code> |
|---------------------------------|---------------------------------|

5. Find the values of `j` and `k` after execution of the following fragment.

```
j = 0;
k = 0;
j = --j;
k = k++;
```

2.4 More Assignment Operators

Java contains a number of assignment operators in addition to the basic one that we have already seen. As an example, a statement of the form

$$\text{<identifier> } += \text{ <expression>;}$$

is equivalent to a statement of the form

$$\text{<identifier> } = \text{ <identifier> } + (\text{<expression>});$$

This can be very useful whenever we want to add a quantity to the value of a variable.

Example 1

- (a) The statement $n += 3;$ is equivalent to $n = n + (3);$
- (b) The statement $x += 2.5;$ is equivalent to $x = x + (2.5);$
- (c) The statement $m += n - 2;$ is equivalent to $m = m + (n - 2);$

Assignment can also be combined with the other four basic arithmetic operators.

$$-= \quad *= \quad /= \quad \%=$$

In each case, the effect is similar to that seen with $+=$, as illustrated in the next example.

Example 2

- (a) The statement `x -= y + 1;` is equivalent to `x = x - (y + 1);`
- (b) The statement `m /= n * p;` is equivalent to `m = m / (n * p);`
- (c) The statement `n *= 5;` is equivalent to `n = n * (5);`

Java permits the use of multiple assignments in one statement. In such statements, the assignment operators are applied from *right to left*, rather than from left to right.

Example 3

The statement

```
i = j = k = 1;
```

first assigns the value 1 to `k`, then the value that has been assigned to `k` is assigned to `j`, and finally the value that has been assigned to `j` is assigned to `i`. Thus, the given statement is equivalent to

```
i = (j = (k = 1));
```

All types of assignment operators can be mixed in a multiple assignment statement. All varieties of assignment operator have the same precedence, lower than that of any other Java operator. As with simple assignments, evaluation again takes place from right to left.

Example 4

Suppose that `i`, `j`, and `k` are `int` variables with values 1, 2, and 3 respectively. Then the statement

```
i /= j -= k + 2;
```

would be evaluated as follows:

```
i /= j = j - (k + 2);
```

```
i /= j = 2 - (3 + 2);
```

```

i /= j = -3;
i = i / (j);
i = 1 / (-3);
i = 0;

```

The final values of `i`, `j`, and `k` would be 0, -3, and 3 respectively.

The next table summarizes the precedence rules for the operators that we have seen so far. Operators with higher precedence appear above those with lower precedence. Parentheses can be used to alter the order of evaluation.

Operator	Operation
<code>++ --</code>	increment, decrement
<code>+ -</code>	unary plus, minus
<code>(<type>)</code>	cast to <code><type></code>
<code>* / %</code>	multiplication, division, remainder
<code>+ -</code>	addition/concatenation, subtraction
<code>= += -= *= /= %=</code>	assignments

In trying to understand how assignment operators behave, it may be helpful to explore the process that occurs in the evaluation of a Java expression. In many ways, assignment operators are like arithmetic operators. An expression containing a mixture of arithmetic and assignment operators will be evaluated using the precedence rules that we have noted previously, with assignments having the lowest precedence. However, when Java evaluates an assignment expression, it also carries out an assignment of a value to a variable. The *value* of an assignment expression is the value that is assigned to the variable. The actual assignment process is simply a *side effect* of the evaluation of the expression.

Example 5

The sequence of statements

```

int i = 4, j = 2;
System.out.println(i += j = 5 + 3 * 4);

```

is perfectly valid.² To perform the `println`, Java must first evaluate its

²Although the code is valid, using a complex and possibly confusing expression like the one shown in the `println` statement is not recommended. In programming, you should always strive to maximize clarity, not to confuse your readers.

argument. The steps in the evaluation of that expression are shown in the following table.

Evaluation Stage	Expression Value	i	j
Start	<code>i += j = 5 + 3 * 4</code>	4	2
Multiply 3 by 4	<code>i += j = 5 + 12</code>	4	2
Add 5 plus 12	<code>i += j = 17</code>	4	2
Assign 17 to j	<code>i += 17</code>	4	17
Assign 21 to i	21	21	17

The value 21 is then converted to a string and printed. As side effects of the evaluation of the expression, the values of `i` and `j` have been set to 21 and 17 respectively.

Exercises 2.4

- Suppose that `x` is a `double` variable whose value is 3.0 just before each statement is executed. Find the value of `x` after execution of the statement.
 - `x *= 2;`
 - `x += x;`
 - `x *= 2/3;`
 - `x /= 15/6;`
- Suppose that, before each statement is executed, the values of the `int` variables `i` and `j` are 4 and 7 respectively. Find their values after execution of the statement.
 - `j += i;`
 - `i *= j;`
 - `i -= j--;`
 - `i *= --j;`
 - `j %= --i;`
 - `j /= ++i;`
 - `i *= ++j + 2;`
 - `j *= i-- % 3;`
- Suppose that, before each statement is executed, the values of the `int` variables `i`, `j`, and `k` are 3, 2, and 1 respectively. Find their values after execution of the statement.

- (a) `i = j = k;` (b) `i = j += k;`
 (c) `i *= j = k + 2;` (d) `i -= j += k++;`
 (e) `i += j *= ++k + 1;` (f) `i %= j = k++ % 4;`

4. State the error in each statement.

- (a) `i+3 += j;` (b) `i *= j - 2 = k;`

2.5 Arithmetic and Characters

We saw in the previous chapter that it was possible to convert values between numerical forms and characters by using an assignment statement, possibly with a cast. It is also possible (and often useful) to do arithmetic involving `char` values.

Before we start doing arithmetic with characters, let us explore some aspects of the relationship in Java between integers and characters. Recall that all Java characters are stored as 16 bits using the Unicode encoding system. As an example, the character `'A'` is stored as the pattern 0000 0000 0100 0001. If we consider this as a base 2 numeral, it has a base 10 value of $1 \times 2^6 + 1 \times 2^0 = 64 + 1 = 65$. If we were to write `int n = 'A';` then Java would assign the value 65 to `n`. The assignment statement copies the bit pattern for `'A'` into the location reserved for `n`. Although the pattern of the bits does not change, the way that Java *interprets* the bit pattern does change — from the `char` value `'A'` to the `int` value 65.³

In Unicode, the letters of the alphabet have been assigned codes for which the numeric values are sequential. Thus the numeric value corresponding to `'B'` is 66, that for `'C'` is 67, and so on. The lower case letter characters: `'a'`, `'b'`, `'c'`, ... also have sequential numeric values (from 97 to 122). Using these properties, we can alter characters using arithmetic operators.

³See Appendix C for more information about representation of numbers using different bases and the relationships between characters and integers.

The ASCII Table

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
00	00	NUL	32	20	SP	64	40	@	96	60	‘
01	01	SOH	33	21	!	65	41	A	97	61	a
02	02	STX	34	22	"	66	42	B	98	62	b
03	03	ETX	35	23	#	67	43	C	99	63	c
04	04	EOT	36	24	\$	68	44	D	100	64	d
05	05	ENQ	37	25	%	69	45	E	101	65	e
06	06	ACK	38	26	&	70	46	F	102	66	f
07	07	BEL	39	27	'	71	47	G	103	67	g
08	08	BS	40	28	(	72	48	H	104	68	h
09	09	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

Example 1

If we have created a `char` value `c` by the declaration

```
char c = 'A';
```

then the statement

```
c++;
```

will change the value of `c` to `'B'`.

We must be very careful to obey Java's type conversion rules when we are doing arithmetic.

Example 2

The statements

```
char c = 'A';  
c = c + 1;
```

will produce an error message because the expression `c + 1` combines a `char` and an `int` to produce an `int` result. The assignment to the `char` variable `c` is an attempt to convert an `int` to a `char` — a narrowing conversion. We can correct the problem by writing the assignment statement in the form

```
c = (char)(c + 1);
```

The numeric values of the encodings for the characters `'0'`, `'1'`, ..., `'9'` are *not* 0, 1, ..., 9 but they *are* sequential values. We can use this fact to convert between the `char` form of a digit and the numerical value of the digit. The next example shows how this can be done.

Example 3

Given the statement

```
char c = '7';
```

then the statement

```
int i = c - '0';
```

will assign to `i` the value 7 because the difference between the encoding of `'7'` and the encoding of `'0'` is 7.

Exercises 2.5

1. State the integer value of each expression.

- | | |
|----------------------------|----------------------------|
| (a) <code>'E' - 'A'</code> | (b) <code>'5' - '0'</code> |
| (c) <code>'m' - 'p'</code> | (d) <code>'3' - '8'</code> |

2. State the value of each expression.

- (a) `(char)(5 + 't')`
 (b) `(char)('2' + 3)`
 (c) `(int)('z' - 'v')`
 (d) `(int)('A' + 'f' - 'D' - 'b')`

3. For each valid fragment, state what it does. If a fragment contains an error, explain the nature of the error and rewrite the fragment with the error corrected.

- | | |
|---|---|
| (a) <code>char a = 'e';</code>
<code>a--;</code> | (b) <code>char b = '4';</code>
<code>b = b - 2;</code> |
| (c) <code>int c = 'g';</code>
<code>--c;</code>
<code>c = c - 'k';</code> | (d) <code>int d = '7';</code>
<code>char dd;</code>
<code>dd = d++;</code> |
| (e) <code>int e = 'q' + 2;</code>
<code>e++;</code>
<code>char ee = e;</code> | (f) <code>char f = 'M';</code>
<code>--f;</code>
<code>int ff = f-- - 'J';</code> |

4. Write a Java program that will first prompt the user for an upper case letter of the alphabet, read the letter supplied by the user, and then print the letter along with its position in the alphabet. For example, if the input is 'D', the program should print

D is at position 4 in the alphabet.

2.6 Using Math Methods

We have already encountered the classes `In` and `System` where we found methods for performing input and output. To assist us in doing mathematics beyond basic arithmetic, Java has a large group of methods contained in a class called `Math`. In this section we will examine some of the most frequently used methods in the `Math` class. To use any of the methods in `Math`, we must identify them as being in that class by including the class name as part of the method name. Each of these methods computes a value; we often speak of this value as the value *returned* by the method.

`Math.abs`

The method `Math.abs` determines the absolute value of an expression. The expression on which `Math.abs` operates is called the *argument* of the method. To use the method, the argument is enclosed in parentheses, just as it is in a function in mathematics. The argument can be of any primitive type other than `boolean`. The type of value returned by the method corresponds to the type of its argument. If the argument is of type `int`, `long`, `float`, or `double`, the method will return a value of that type. If the argument is of a type narrower than `int`, the method will return a value of type `int`.⁴

Example 1

- (a) `Math.abs(-4) ⇒ 4`
- (b) `Math.abs(2.7f) ⇒ 2.7f`
- (c) `Math.abs(-8.2e-4) ⇒ 8.2e-4`
- (d) `Math.abs(-456L) ⇒ 456L`

⁴See Chapter 5 for a more detailed discussion of arguments.

Math.sqrt

The `Math.sqrt` method returns the positive square root of the argument given to it. The method will take a numerical value of any type as argument. It returns a value of type `double`. If the method is given a negative value, it will return the value `NaN` (Not a Number).

Example 2

- (a) `Math.sqrt(4) ⇒ 2.0`
- (b) `Math.sqrt(7.0) ⇒ 2.6457513110645907`
- (c) `Math.sqrt(-5.0) ⇒ NaN`

Math.pow

To calculate small positive powers of numbers, it is most efficient to simply use the multiplication operator, `*`. In addition, to calculate $x^{0.5} = \sqrt{x}$, it is best to use the `Math.sqrt` method. However, to calculate other powers, we can use the method `Math.pow` that will evaluate x^y . The method always returns a `double` value, no matter what types the arguments are. A call of the form `Math.pow(a,b)`, if `a` and/or `b` are not `double` values, will first determine the `double` equivalent of `a` and `b`. It will then determine and return the value of `ab` as a `double` value. If the exponent is not an integer and the base is negative, the method returns `NaN`.

Example 3

- (a) `Math.pow(8,1.0/3.0) ⇒ 2.0`
- (b) `Math.pow(2,10) ⇒ 1024.0`
- (c) `Math.pow(-32.0,0.2) ⇒ NaN`
- (d) `(int)Math.pow(100,0.25) ⇒ 3`

Math.max, Math.min

The method `Math.max` takes two arguments and returns the larger of the two. As with `Math.abs`, the arguments to `Math.max` can be of type `int`, `long`, `float`, or `double`. If the two arguments are of the same type, the method will return that type; if they are of different types, the method will return a value of the wider type. The method `Math.min` returns the minimum of two values. It operates in a way exactly analogous to that described for `Math.max`.

Example 4

- (a) `Math.max(-3,2) ⇒ 2`
- (b) `Math.max(3.4f,5.3f) ⇒ 5.3f`
- (c) `Math.max(7,3L) ⇒ 7L`
- (d) `Math.max(-9e-4,-1.1e-6) ⇒ -1.1e-6`
- (e) `Math.min(8,7) ⇒ 7`
- (f) `Math.min(1.5f,2.0) ⇒ 1.5`
- (g) `Math.min(-4.7,-9.3) ⇒ -9.3`
- (h) `Math.min(18,6.7) ⇒ 6.7`

Math.round, Math.ceil, Math.floor

The `Math.round` method rounds a floating point value to the nearest integer. It returns an `int` value if its argument is `float` and returns a `long` value if its argument is `double`. The `Math.ceil` and `Math.floor` methods both take a `double` argument and return a `double` value. `Math.ceil` returns a `double` that has the smallest integer value that is not less than its argument while `Math.floor` returns a `double` that has the largest integer value that is not greater than its argument.

Example 5

- (a) `Math.round(2.7)` $\Rightarrow$ 3L (b) `Math.round(4.5f)` $\Rightarrow$ 5
 (c) `Math.round(-2.13f)` $\Rightarrow$ -2 (d) `Math.floor(21.94)` $\Rightarrow$ 21.0
 (e) `Math.floor(8.0)` $\Rightarrow$ 8.0 (f) `Math.ceil(18.8)` $\Rightarrow$ 19.0
 (g) `Math.ceil(-5.47)` $\Rightarrow$ -5.0 (h) `Math.ceil(-5.0)` $\Rightarrow$ -5.0

The `Math.round` method can be adapted to rounding to a precision other than the nearest integer. To do this, we can first apply a scaling factor to the original value, round the result to the nearest integer, and then reverse the effect of the scaling.

Example 6

The following program reads and rounds a `double` value to the nearest hundredth, a precision specified by the constant value of that name.

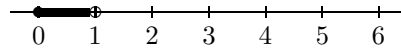
```
class Roundoff
{
    public static void main (String[] args)
    {
        final double PRECISION = 0.01;    // rounding precision
        double inValue;                    // value to be rounded
        double roundedValue;                // value after rounding

        System.out.println("Please supply a value for rounding");
        inValue = In.getDouble();
        roundedValue = Math.round(inValue/PRECISION)*PRECISION;
        System.out.println("Input value:  " + inValue);
        System.out.println("Rounded value: " + roundedValue);
    }
}
```

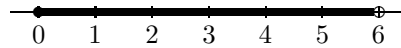
Math.random

The `Math.random` method takes no argument. It returns a `double`, a random value x in the interval $0 \leq x < 1$. By combining this method with other methods and operations, we can generate random values suitable for a wide variety of applications. As an example, suppose that we wish to simulate the throwing of a *fair* die — one for which each of the six faces on the die is equally likely to turn up. We can do so by performing the following operations.

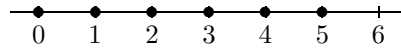
We can begin by calling the method `Math.random` to generate a value distributed randomly in the interval $0 \leq x < 1$.



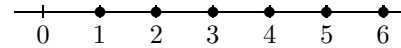
Multiplying this by 6, we expand the range to get a value distributed randomly in the interval $0 \leq x < 6$.



Next, casting this result as an `int`, we chop off any fractional part to obtain a random integral value in the range $0, 1, \dots, 5$.



Finally, adding one gives us a random integral value in the desired range: $1, 2, \dots, 6$.



All of these operations can be performed in one step with the statement

```
int dieRoll = (int)(6 * Math.random()) + 1;
```

Constants

In addition to its methods, the `Math` class also has two important `double` constants: `Math.PI` and `Math.E`. The constant `Math.PI` gives the value of $\pi = 3.14159\dots$ to 16 significant digits while `Math.E` gives the value of $e = 2.71828\dots$ to 16 significant digits.

Trigonometric Methods

The class has a number of trigonometric methods, including `Math.sin`, `Math.cos`, `Math.tan`, `Math.asin`, `Math.acos`, and `Math.atan`. Java, like

most programming languages, prefers to work in radians rather than degrees. To convert from one measurement system to another, you can use the relationship

$$\pi \text{ radians} = 180 \text{ degrees}$$

The methods `Math.sin`, `Math.cos`, and `Math.tan` each take a single argument, representing the radian measure of an angle. They return the sine, cosine, and tangent of their arguments, as `double` values.

Example 7

- (a) $\sin 60^\circ = 0.866025\dots$ `Math.sin(Math.PI/3)  $\Rightarrow$  0.866025...`
- (b) $\cos 5^\circ = 0.996194\dots$ `Math.cos(5*Math.PI/180)  $\Rightarrow$  0.996194...`
- (c) $\tan 45^\circ = 1$ `Math.tan(Math.PI/4)  $\Rightarrow$  1.0`
- (d) $\cos 180^\circ = -1$ `Math.cos(Math.PI)  $\Rightarrow$  -1.0`
- (e) $\cot 0^\circ$ is not defined `1/Math.tan(0)  $\Rightarrow$  Infinity`

The methods `Math.asin`, `Math.acos`, and `Math.atan` determine values of the inverse trigonometric functions: $\arcsin(\sin^{-1})$, $\arccos(\cos^{-1})$, and $\arctan(\tan^{-1})$. The method `Math.asin` returns, as a `double` value, the measure in radians of an angle whose sine is the argument. The others behave similarly.

Example 8

- (a) $\arccos 0 = \frac{\pi}{2}$ `Math.acos(0)  $\Rightarrow$  1.5707963...`
- (b) $\arctan 1 = \frac{\pi}{4}$ `Math.atan(1)  $\Rightarrow$  0.7853981...`
- (c) $\arcsin \frac{\sqrt{3}}{2} = \frac{\pi}{3}$ `Math.asin(Math.sqrt(3)/2)  $\Rightarrow$  1.047197...`

Exponential and Logarithmic Methods

The class also contains the methods `Math.exp` for determining e^x , and `Math.log` for determining $\ln x$, the logarithm of x to the base e .

Example 9

(a) $e = 2.7182818\dots$	<code>Math.exp(1) ⇒ 2.7182818...</code>
(b) $\ln e = 1$	<code>Math.log(Math.E) ⇒ 1.0</code>
(c) $e^3 = 20.0855369\dots$	<code>Math.exp(3) ⇒ 20.0855369...</code>
(d) $\ln 1 = 0$	<code>Math.log(1.0) ⇒ 0.0</code>
(e) $\ln(-1)$ is not defined	<code>Math.log(-1.0) ⇒ NaN</code>

This section has not given a complete listing of the methods of Java's `Math` class. A full list with notes on all the elements of the class can be found at Sun's web site (<http://java.sun.com/>). At the site, to locate information on any feature of the `Math` class, follow links first to the *Application Programming Interface* or *API*. The API is a collection of hundreds of classes that add functionality of various kinds to Java. The classes of the API are organized into groups called *packages*. The `Math` class is in the package called `java.lang` (not in the package `java.math`). Once you are at the index page of the API, you can either locate a class directly by its class name or find it indirectly by first going to its package and then finding the class within the package. At the time that this was written, the page containing the index of the API for Java 2 was located at <http://java.sun.com/j2se/1.4/docs/api/>.

Now that we can do mathematics in Java, we can write many useful programs. The next example illustrates a program structure that we will see frequently. The program obtains *input*, does some *calculation*, and produces some *output*.

Example 10

For a simple pendulum, the length of time for one swing (the *period*) is determined by the pendulum's length and the force of gravity. If we assume that gravity is constant anywhere on the surface of the earth, then the period of a pendulum is determined by its length. The formula connecting period and length is

$$P = 2\pi\sqrt{\frac{L}{g}}$$

where the period, P , is measured in seconds, the length, L , is measured in metres, and g is the acceleration due to gravity — 9.8 m/s^2 . The following program uses this formula to determine the period of a pendulum of a given length.

```
class Pendulum
{
    public static void main (String[] args)
    {
        final double G = 9.8; // acceleration due to gravity

        System.out.println("Give pendulum length in metres");
        double length = In.getDouble();

        // find period, rounded to nearest tenth of a second
        double period = 2*Math.PI*Math.sqrt(length/G);
        period = Math.round(10*period)/10.0;

        System.out.println("For a length of " + length + " m,"
                           + " pendulum's period is " + period + " s.");
    }
}
```

Exercises 2.6

1. State the value and type of each expression.

(a) `Math.abs(-5)-Math.abs(-7)`

- (b) `Math.abs(-1e-1)+Math.abs(-2e-2)`
- (c) `Math.sqrt(0.0064)`
- (d) `Math.sqrt(Math.pow(2.7,2))`
- (e) `Math.round(3.499)` (f) `Math.max(1.5e-2,0.095)`
- (g) `Math.ceil(4.002)` (h) `Math.min(-5,1.0)`
- (i) `Math.floor(7.99)` (j) `Math.ceil(-2.73)`
- (k) `Math.pow(16,0.25)` (l) `Math.pow(4,-2)`
- (m) `Math.round(1.49 + 0.1)` (n) `Math.round(1.49) + 0.1`

2. Write as Java expressions.

- (a) $\sqrt{a^2 - b^2}$ (b) $\pi(x^6 - y^6)$
- (c) $\frac{4}{3}\pi r^3$ (d) $|z^4 - 1|$
- (e) $\ln|1 + x|$ (f) $x^2 e^x$

3. A student incorrectly attempted to produce a random value uniformly distributed over the set $\{1, 2, \dots, 6\}$ using the expression

`6*(int)Math.random() + 1`

State and explain the actual value of the expression.

4. Write a statement that will make the `int` variable `result` take on a random value uniformly distributed over the given set.

- (a) $\{1, 2, 3, \dots, 10\}$
- (b) $\{1, 2, 3, \dots, 52\}$
- (c) $\{5, 10, 15, \dots, 100\}$
- (d) $\{-5, -4, -3, \dots, 5\}$
- (e) $\{100, 110, 120, \dots, 300\}$
- (f) $\{a, a + b, a + 2b, \dots, a + kb\}$ where a , b , and k are integers.

5. Write a statement that will assign the `char` variable `randChoice` a random value from the set $\{'A', 'B', 'C', 'D', 'E'\}$.

6. Write a statement that will assign the `double` variable `randVal` a random value from the set $\{1.00, 1.25, 1.50, \dots, 4.00\}$.

7.
 - (a) Write a statement that will assign the `int` variable `dodecaRoll` a random value that could result from the rolling of a fair twelve-faced die with faces numbered from one to twelve.
 - (b) Write a statement that will assign the `int` variable `doubleRoll` a random value that could result from noting the sum obtained by rolling a pair of standard, fair dice with faces numbered from one to six.
8. Find the value of each expression.
 - (a) `Math.sin(Math.PI/6)`
 - (b) `Math.cos(Math.PI/3)`
 - (c) `Math.cos(0)`
 - (d) `Math.atan(0)`
 - (e) `Math.tan(Math.atan(0.5))`
 - (f) `Math.asin(Math.sin(1))`
9. Write Java expressions for each of the following.
 - (a) $\sin 45^\circ$
 - (b) $\cos 120^\circ$
 - (c) $\tan 24^\circ$
 - (d) $\sec 18^\circ$
 - (e) $\csc 160^\circ$
 - (f) $\cot 15^\circ$
 - (g) $\arctan 2$
 - (h) $\arcsin 0.5$
 - (i) $\cot^{-1}\sqrt{3}$
 - (j) $\sec^{-1}1.5$
10. Write an expression that could be used to find the value of $\log_{10} x$.
11. In this section, we showed a program that rounded a value to the nearest hundredth. How would that program have to be modified to round values to
 - (a) the nearest tenth?
 - (b) the nearest thousand?
12. Airport runways are often given numbers determined by the direction in which planes travel as they move along the runways. The number of a runway is found by taking the bearing in degrees (to the nearest ten degrees) and dropping the final zero. For example, a runway with a bearing between 265° and 275° would have a runway number of 27. Write a program that asks the user for a bearing (from 0° to 360°) and then determines the corresponding runway number.
13. Write a program that asks for a length of time in hours and then converts this measurement to hours, minutes, and seconds. For example, input of 38.47 should produce output similar to the following:

38.47 hours = 38 hours, 28 minutes, 12 seconds

with some text that identifies the source of the value being printed. For example, to trace the value of the variable `sum` at some point, you could insert the following tracing statement in your program.

```
System.out.println("At A, sum = " + sum);
```

The phrase “at A” identifies the location in the program. If `sum` changes value at some other point, you may want to insert another tracing statement that prints `At B, sum = ...`.

2. Many systems have *debuggers* — programs that can be used to trace variables easily. The operation of debuggers varies widely but any debugger should have facilities to
 - stop a program at a specified point and then restart it
 - display the values of selected variables at the point at which a program was stopped
 - step through a program one statement at a time

If your system does have a debugger, it is well worth your while to learn how to use at least the basic features.

Exercises 2.7

1. Evaluate.
 - (a) $6 / 4 * 2$
 - (b) $2 * 3 / 2$
 - (c) `(int)2.7*1.8`
 - (d) `(int)2.7*(int)1.8`
 - (e) `(int)(2.7*1.8)`
2. For each expression, determine its mathematical value and then run a Java program that prints the value of the expression. Explain any differences.
 - (a) $3.0 * (1.0 / 3.0 + 100.0 - 100.0)$
 - (b) $(5E305 + 7E307) * 10$
 - (c) $18 + 1E18 - 1E18$
 - (d) $1E18 - 1E18 + 18$

(e) `1E-200 * 1E-200 * 1E200 * 1E200`

(f) `1E200 * 1E200 * 1E-200 * 1E-200`

3. A quadratic equation of the form $ax^2 + bx + c = 0$ has roots

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Write a fragment that *efficiently* determines the values of the roots (`root1` and `root2`) of the quadratic equation with coefficients `a`, `b`, and `c`. Assume that all the variables have been declared as type `double` and that the equation has two real roots.

4. Rewrite the statement

```
m = n-- * ++p;
```

using three statements.

2.8 Review Exercises 2

1. Evaluate. Use a decimal point in your answer if the result is `double`.

(a) `15 / 6 + 15 % 6`

(b) `3e-1 + 2e1`

(c) `2 / 5 * 8.0`

(d) `1.6 * 20 % 8`

(e) `(double)(25/4)`

(f) `(int) 2.7 + 6.3`

(g) `20 - 10*(15%4)`

(h) `7/(-5) + 4%(-3)`

(i) `2*3/(double)4`

(j) `(int) 4.8 % 1.1`

2. Each of the following expressions is intended to evaluate the expression

$$\frac{ax^2 + b}{cx + d}$$

Some are correct while others are not. Classify each as being correct or incorrect. For those that are incorrect, give the reason(s).

(a) `(a * x * x + b) / ((c * x + d)`

(b) `(a * Math.pow(x,2) + b) / (c * x + d)`

- (c) $((a)(x)(x) + (b)) / ((c)(x) + d)$
 (d) $(b + x * (x * (a))) / (d + x * (c))$
 (e) $(a) * \text{Math.pow}(x, 2) + (b) / ((c) * (x) + d)$
 (f) $(a * (x * x)) + (b) / c * (x) + d$

3. Suppose that the following declarations have been made.

`int i = 3, j = 4, k = 2;`

Using these starting values in each part, find the value of each variable after the given statement has been executed.

- (a) `j = ++i * k--;` (b) `i = --j + k/2;`
 (c) `k = i-- - j++;` (d) `j = (2*i++)%(++k + 1);`
 (e) `i += j -= --k;` (f) `i *= j /= k++;`

4. State the value of each expression.

- (a) `(char)('b' + 6)` (b) `(int)('M' - 'T')`

5. Evaluate.

- (a) `Math.round(Math.sqrt(20))`
 (b) `Math.ceil(-4.6)`
 (c) `Math.min(0.0024, 1.2e-3)`
 (d) `Math.pow(0.5, -4)`

6. Write as Java expressions.

- (a) $\sqrt{\pi|a^2 - b^2|}$ (b) $\sqrt[4]{x^6 + y^6}$
 (c) $\frac{-b + \sqrt{b^2 - 4ac}}{2a}$ (d) $\frac{1}{a^{-1} + b^{-1}}$

7. Write as Java statements.

- (a) $y = \frac{1}{\sin^2 x - \cos^2 x}$ (b) $s = \sqrt{1 + \tan^2 x}$
 (c) $t = \ln |1 - x^4|$ (d) $y = \frac{1}{\sqrt{2\pi}} e^{-x^2}$

8. To switch the values contained in the variables `x` and `y`, a programmer wrote the following segment:

```
x = y;
```

```
y = x;
```

- (a) If, before execution of the segment, `x` contained the value 7 and `y` contained the value 4, what value would each have after the segment was performed?
- (b) Rewrite the segment so that it performs the intended task correctly.
9. Write a program that reads three `double` values and computes their mean, rounded to two decimal places.
10. The current deficit of Arrakis is 47 000 000 grods and it is estimated that the deficit will increase by 4.5% in each of the next two years. Write a program that will find the estimated deficit in these two years, rounded correctly to the nearest million grods.
11. Write a program that asks the user for a three-digit number, finds the sum of the digits of the number, and then prints both the number and its digit sum.

Projects

12. As you are probably aware, the date of Easter can vary widely from year to year. The Council of Nicæa in the year 325 decreed that Easter should be held on the first Sunday following the full moon that occurs on or after March 21, the usual day of the vernal equinox. Because the date depends on solar, lunar, and calendar cycles, it is not easy to find an arithmetic procedure that determines the correct date for any given year. The one given below (for the Gregorian calendar) was created by J. M. Oudin in 1940. In the equations, y represents the year, m represents the month (3 for March and 4 for April), and d represents the day of the month. All variables are

integers; any remainders produced by divisions should be discarded.

$$\begin{aligned}
 p &= \frac{y}{100} \\
 q &= y - 19 \left(\frac{y}{19} \right) \\
 r &= \frac{p - 17}{25} \\
 s &= p - \frac{p}{4} - \frac{p - r}{3} + 19q + 15 \\
 s &= s - 30 \left(\frac{s}{30} \right) \\
 s &= s - \left(\frac{s}{28} \right) \left[1 - \left(\frac{s}{28} \right) \left(\frac{29}{s+1} \right) \left(\frac{21-q}{11} \right) \right] \\
 t &= y + \frac{y}{4} + s + 2 - p + \frac{p}{4} \\
 t &= t - 7 \left(\frac{t}{7} \right) \\
 u &= s - t \\
 m &= 3 + \frac{u + 40}{44} \\
 d &= u + 28 - 31 \left(\frac{m}{4} \right)
 \end{aligned}$$

Write a program that asks the user for a year and then gives the month and day of Easter in that year.

13. In order for an object to escape a planet's gravitational pull, it must attain a minimum initial velocity called the *escape velocity*. The escape velocity varies from planet to planet but it is the same for all objects on a given planet. Assume that we are analyzing the data that a small probe has collected while exploring some mystery planet. The probe has managed to obtain the circumference of the planet and the acceleration due to gravity at the surface. The probe must now determine what initial velocity it requires for takeoff in order to remove itself from the planet's gravitational force. You are to create a Java program that will determine this velocity. Your program should first prompt the user for the circumference of the planet and then the acceleration due to gravity on the planet. From this information your program should determine the radius, mass, and escape velocity

of the planet using the following equations.

$$v_{\text{escape}} = \sqrt{\frac{2Gm}{r}} \quad a = \frac{Gm}{r^2}$$

In these equations, m kg is the planet's mass, r km is the planet's radius, G is the gravitational constant approximated by $6.6726 \times 10^{-11} \text{m}^3 \text{kg}^{-1} \text{s}^{-2}$, and a m/s² is the acceleration due to gravity on the surface of the planet. Your program should read input and print output as shown in the following example. (The values shown after the question marks are supplied by the user.)

```
Circumference (km) of planet? 38000
Acceleration due to gravity (m/s^2) on planet? 9.8

Calculating the escape velocity...
Planet radius: 6047.9 km
Planet mass: 5372.0 x 10^21 kg
Escape velocity: 10.9 km/s
```

Note: All results are rounded to one decimal place for printing. Your program must handle the unit conversions as well as printing the planet's mass in terms of 10^{21} kg. Assume that this will always produce reasonable results. Assume that the planet is perfectly spherical.